MODUL PRAKTIKUM API & BASIS DATA

Tingkat: Menengah | Durasi: 100 Menit | Target Output: API Terkoneksi Database

1. Tujuan Praktikum

- Mampu memecah kode ke dalam modul (Router) agar struktur proyek rapi.
- Mampu menghubungkan API ke MySQL dan membuat validasi input dengan Pydantic.
- Mampu mengubah Query SQL dari Modul 2 menjadi endpoint API (GET, POST, DELETE).

2. Refactoring & Struktur Folder

Buatlah struktur folder seperti di bawah ini, dan pastikan Anda sudah menginstal library: `pip install mysql-connector-python pydantic email-validator`.

```
/proyek_api
├── main.py            # Entry point aplikasi (Mendaftarkan Router)
├── database.py        # Menyimpan logika koneksi DB
└── routers/
    └── mahasiswa.py   # Semua endpoint khusus Mahasiswa ada di sini
```

Isi file `database.py`:

```
import mysql.connector

def get_db():
    return mysql.connector.connect(
        host="localhost", user="root", password="", database="db_kampus"
    )
```

3. Model Validasi (Pydantic)

Di bagian atas file `routers/mahasiswa.py`, definisikan model Pydantic. Pydantic bertugas sebagai "satpam" yang memastikan format JSON dari client sudah benar sebelum masuk ke database.

```
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel, Field, EmailStr
from database import get_db

router = APIRouter(prefix="/mahasiswa", tags=["Mahasiswa"])

class MahasiswaInput(BaseModel):
    nim: str = Field(..., min_length=4, max_length=10)
    nama: str = Field(..., min_length=3)
    email: EmailStr
```

4. Membuat Endpoint GET (Dengan JOIN)

Lanjutkan di file `routers/mahasiswa.py`. Kita ambil query JOIN dari Modul 2 untuk dieksekusi di API.

```
@router.get("/")
def get_semua_mahasiswa():
    db = get_db()
    cursor = db.cursor(dictionary=True) # dictionary=True mereturn JSON object

    sql = """
        SELECT m.nim, m.nama, p.nama_prodi
        FROM mahasiswa m
        JOIN prodi p ON m.id_prodi = p.id_prodi
    """
    cursor.execute(sql)
    result = cursor.fetchall()
    db.close()

    return {"data": result}
```

5. Endpoint POST & Parameterized Query

Untuk mencegah *SQL Injection*, jangan pernah menggabungkan string variabel secara langsung. Selalu gunakan penanda `%s`.

```
@router.post("/", status_code=201)
def tambah_mahasiswa(mhs: MahasiswaInput):
    db = get_db()
    cursor = db.cursor()

    sql = "INSERT INTO mahasiswa (nim, nama, email, id_prodi) VALUES (%s, %s, %s, %s)"
    val = (mhs.nim, mhs.nama, mhs.email, mhs.id_prodi)

    try:
        cursor.execute(sql, val)
        db.commit() # Simpan permanen ke database
    except Exception as e:
        db.rollback()
        raise HTTPException(status_code=400, detail=str(e))
    finally:
        db.close()

    return {"message": "Data berhasil disimpan"}
```

> **Menjalankan Aplikasi dari `main.py`:**
> Di file `main.py`, hubungkan router yang baru dibuat:
> `from fastapi import FastAPI`
> `from routers import mahasiswa`
> `app = FastAPI()`
> `app.include_router(mahasiswa.router)`
> Jalankan dengan `uvicorn main:app --reload` dan cek di Swagger!